20 JavaScript Interview Questions

> Next.js SaaS Boilerplate & Full-Stack Dev Course
>
> ➡️ www.thedevspace.io

✅ Core JavaScript

1. **What is the difference between `var`, `let`, and `const`?**
   - `var`: Function-scoped, hoisted, can be redeclared.
   - `let`: Block-scoped, not hoisted the same way, can't be redeclared in the same scope.
   - `const`: Block-scoped, must be initialized, value cannot be reassigned.

---

2. **How does JavaScript handle asynchronous code?**

- Through **callbacks**, **promises**, and **async/await**.
- The **event loop** and **task queue** ensure non-blocking behavior.

---

3. **What are closures in JavaScript? Give an example.**

- A closure is a function that remembers the variables from its lexical scope even when executed outside that scope.

```
function outer() {
  let count = 0;
  return function inner() {
    return ++count;
  }
}
const counter = outer();
counter(); // 1
counter(); // 2
```

---

4. **Explain the concept of hoisting in JavaScript.**
   - Declarations (`var`, `function`) are moved to the top of their scope during compilation.
   - `let` and `const` are hoisted but not initialized (temporal dead zone).

---

5. **What is the `this` keyword? How does it behave in different contexts?**
   - Refers to the object calling the function.
   - In global context: `window` (in browsers).
   - In strict mode: `undefined`.
   - In arrow functions: lexically bound (doesn't change based on caller).

---

6. **What is the difference between `==` and `===` in JavaScript?**
   - `==`: loose equality, allows type coercion.
   - `===`: strict equality, checks type and value.

---

7. **What are truthy and falsy values in JavaScript?**
   - **Falsy values**: `false`, `0`, `''`, `null`, `undefined`, `NaN`.
   - Everything else is truthy.

---

8. **How does the event loop work in JavaScript?**
   - Executes **synchronous code** first, then handles **asynchronous tasks** via the **call stack**, **callback queue**, and **microtask queue** (promises first, then other callbacks).

---

9. **What are arrow functions, and how do they differ from regular functions?**
   - Shorter syntax.
   - Do **not** have their own `this`, `arguments`, `super`, or `new.target`.
   - Cannot be used as constructors.

0. **What is the difference between shallow copy and deep copy?**

- **Shallow copy** copies references to nested objects.
- **Deep copy** duplicates all levels of the object.

```
// Shallow
const copy = {...original};
// Deep
const deepCopy = JSON.parse(JSON.stringify(original));
```

✅ Object-Oriented and Functional Programming

1. **How does prototypal inheritance work in JavaScript?**

- Objects inherit properties from their prototype chain via `__proto__` or `Object.create()`.
- Functions have a `prototype` object used for inheritance when using `new`.

---

2. **What are higher-order functions? Give an example.**

- Functions that take other functions as arguments or return them.

```
function multiplyBy(n) {
  return function(x) {
    return x * n;
  };
}
const double = multiplyBy(2);
```

---

3. **What is the difference between call, apply, and bind?**

- `call`: invokes a function with a given `this` and arguments.
- `apply`: same as `call`, but args passed as an array.

- `bind`: returns a new function with `this` bound.

---

4. **How do you implement private variables in JavaScript?**

- Using closures or `#` private fields in classes.

```
class Counter {
  #count = 0;
  increment() { this.#count++; }
  get value() { return this.#count; }
}
```

---

5. **What are the advantages of functional programming patterns in JavaScript?**

- Predictable, testable, less side-effect prone.
- Encourages immutability and composability.

✅ Browser and DOM

6. **How does event delegation work in JavaScript?**

- Attaching a single event listener to a parent instead of each child.
- Uses **event bubbling** to catch events from children.

---

7. **What is the difference between `e.preventDefault()` and `e.stopPropagation()`?**

- `preventDefault()`: stops the default browser action.
- `stopPropagation()`: stops the event from bubbling up the DOM.

---

8. **How do you debounce or throttle a function in JavaScript?**

- **Debounce**: delays execution until a pause in input.
- **Throttle**: limits execution to once per time interval.

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

✅ ES6+ and Modern JS

9. **What are destructuring and spread/rest operators in JavaScript?**

- **Destructuring**: extracting values from arrays/objects.

```
const [a, b] = [1, 2];
const {name} = {name: 'Eric'};
```

- **Spread (`...`)**: expands iterable values.

```
const arr = [...arr1, ...arr2];
```

- **Rest (`...`)**: collects remaining values.

```
function sum(...args) {}
```

---

0. **What are modules in JavaScript? How do `import` and `export` work?**

- Modules split code into reusable files.
- Use `export` to expose parts of a module and `import` to bring them into another.

```
// module.js
export function greet() {}

// main.js
import { greet } from './module.js';
```